02 AGOSTO 2024

IMPLEMENTAÇÃO DE CIRCUITOS COMBINACIONAIS COM MULTIPLEXADORES

Experimento 5

GRUPO D8:

Mariana Simion
Rodrigo Rafik
Rute Alves

UnB | CIC

RESUMO

Neste relatório, exploramos o projeto e a simulação de um somador completo utilizando multiplexadores. Inicialmente, criamos um MUX 8:1 em linguagem de descrição de hardware Verilog, utilizando dois multiplexadores: um para cada saída do circuito. Cada multiplexador teve suas entradas de dados definidas de acordo com os mintermos de suas respectivas saídas. Em seguida, realizamos a simulação por forma de onda com um período de 20ns para verificar o comportamento do somador em relação às suas saídas esperadas. Posteriormente, criámos um novo MUX 8:1 em Verilog com a sua descrição comportamental, descrevendo todas as combinações das entradas para as quais as saídas do circuito fossem iguais a 1. Realizamos simulações funcionais e temporais do circuito completo, verificando a tabela verdade e observando os atrasos impostos pelo circuito. Por fim, transferimos o circuito para a placa FPGA, onde pudemos testá-lo na prática.

ABSTRACT

In this report, we explore the design and simulation of a full adder using multiplexers. Initially, we created an 8:1 MUX in the Verilog hardware description language, using two multiplexers: one for each circuit output. Each multiplexer had its data inputs defined according to the minterms of their respective outputs. Then, we performed the waveform simulation with a period of 20ns to verify the behavior of the adder in relation to its expected outputs.
Subsequently, we created a new 8:1 MUX in Verilog with its behavioral description, describing all combinations of inputs for which the circuit's outputs were equal to 1. We performed functional and temporal simulations of the complete circuit, checking the truth table and observing delays imposed by the circuit. Finally, we transferred the circuit to the FPGA board, where we were able to test it in practice.

INTRODUÇÃO

Os multiplexadores, também chamados de MUX ou seletores, são um tipo de circuito combinacional que recebem diversas entradas e selecionam uma delas para a saída. Esse funcionamento é chamado de multiplexação. Este circuito é composto por três tipos de terminais: os de seleção, os de entrada de dados e o de saída. Os terminais de seleção determinam qual terminal de entrada de dados terá seu conteúdo transmitido para o terminal de saída. Em contraste, os demultiplexadores fazem o exato oposto. Os demultiplexadores, também chamados de DEMUX, são um tipo de circuito combinacional que selecionam uma saída para a qual será enviado o valor da entrada. Este circuito também é composto por três tipos de terminais: os de endereçamento, o de entrada de dado e os de saída. Os terminais de endereçamento determinam para qual terminal de saída o dado de entrada será transmitido.

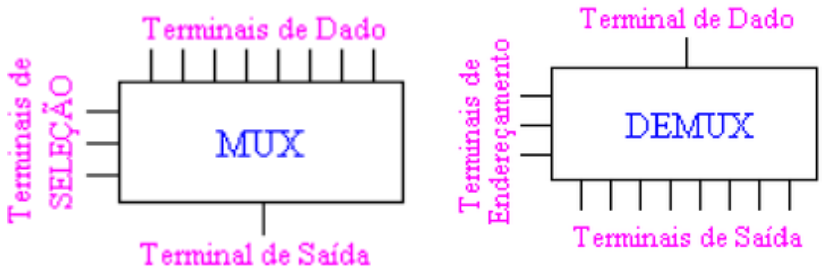

Figura 1. Multiplexador e demultiplexador

Os multiplexadores e demultiplexadores podem ser implementados a partir de portas lógicas. No entanto, utilizando a linguagem de descrição de hardware (HDL) Verilog, a implementação desses circuitos se torna muito mais simples, como é mostrado na figura 2.

```
module mux2(
        input [1:0] Dado,
        input Escolha,
        output Saida
);

 assign Saida = Dado[Escolha];

endmodule
```

```
module demux2(
        input Dado,
        input Escolha,
        output [1:0] Saida
        );

 assign Saida[0]= Escolha==1'b0 ? Dado : 1'b0;
 assign Saida[1]= Escolha==1'b1 ? Dado : 1'b0;

endmodule
```

Figura 2. Implementação de um MUX 2:1 e um DEMUX 1:2 em Verilog

Com base nesses conceitos, o experimento realizado em laboratório tem o intuito de aplicar de forma prática o funcionamento dos multiplexadores e demultiplexadores, além de introduzir a linguagem de descrição de hardware Verilog. Para a realização deste experimento, foram utilizados o software de desenvolvimento Quartus II e a placa FPGA DE2.

PROCEDIMENTOS E RESULTADOS

No item 2.1, projetamos e simulamos um somador completo utilizando dois multiplexadores 8:1. Na etapa 2.2, utilizando o circuito do item anterior, realizamos sua implementação em Verilog, simulamos e, por fim, transferimos para a placa FPGA.

2.1 Somador Completo com MUX 8:1

Na primeira parte do experimento, projetamos e simulamos um somador completo utilizando dois multiplexadores 8:1. Inicialmente, utilizando a linguagem de descrição de hardware Verilog, implementamos um MUX 8:1.

```
1  module mux8(
2      input [7:0] Dado,
3      input [2:0] Escolha,
4      output Saida
5  );
6  assign Saida = Dado[Escolha];
7  endmodule
8
```

Figura 3. Multiplexador 8:1 em Verilog

Em seguida, criamos um bloco utilizando o MUX 8:1 e, com base na tabela verdade fornecida pelo roteiro do experimento, desenhamos o diagrama esquemático do circuito. Foram necessários dois multiplexadores, um para a saída S e outro para a saída Cout. Cada multiplexador teve suas entradas de dados definidas de acordo com os mintermos de suas respectivas saídas, e todas as conexões foram realizadas utilizando VCC como 1 e GND como 0.

Entradas			Saídas	
A	B	Cin	Cout	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Tabela 1. Tabela verdade do somador completo

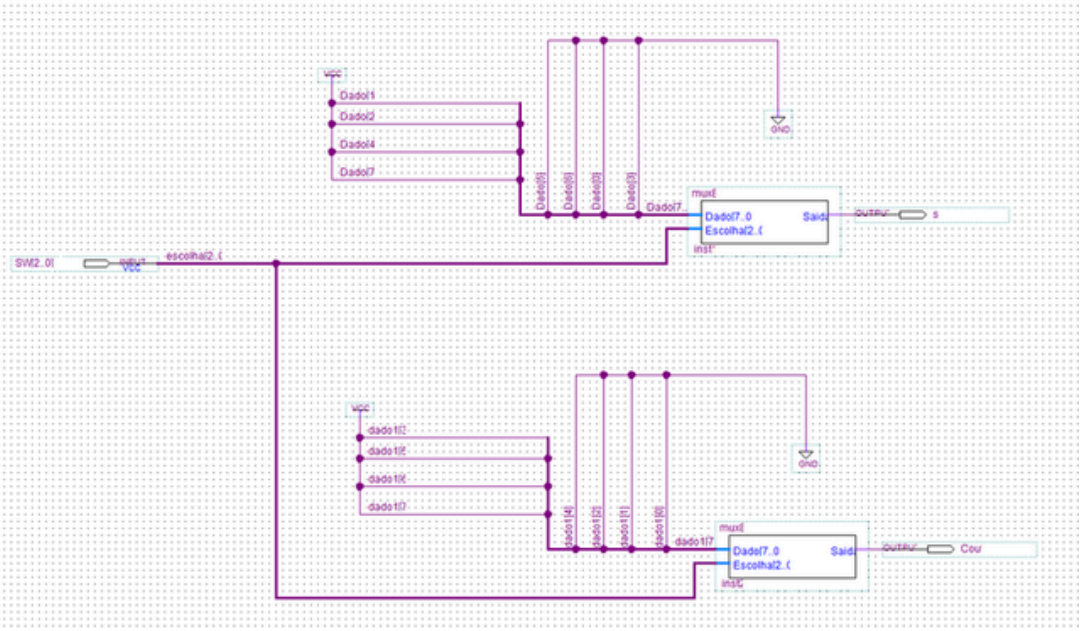

Figura 4. Diagrama esquemático do somador completo

Realizamos a compilação do circuito e obtivemos resultados positivos. Após essa etapa, com o circuito já compilado e sem erros, realizamos a simulação funcional em forma de onda para verificar o funcionamento lógico do mesmo.

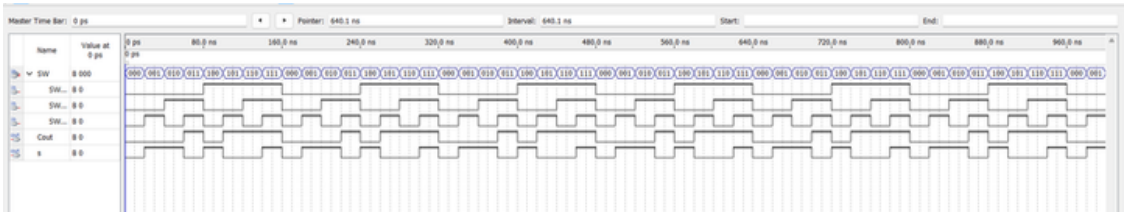

Figura 5. Simulação funcional em forma de onda do somador completo

A partir da simulação funcional, foi possível verificar que a lógica do circuito está funcionando corretamente, pois os resultados obtidos estão coerentes com a tabela verdade fornecida. Após essa etapa, realizamos a simulação temporal do circuito.

Figura 6. Simulação temporal em forma de onda do somador completo

A partir da simulação temporal, podemos perceber que, apesar da lógica do circuito funcionar corretamente, há um pequeno atraso nas formas de onda e a presença de hazards. Essas anomalias ocorrem devido ao atraso das portas lógicas, que causam transições temporárias indesejadas, fazendo com que a saída, enquanto deveria ser 0, ocasionalmente seja 1, e vice-versa.

2.2 Somador Completo em Verilog

Na segunda etapa do experimento, realizamos a descrição comportamental do somador completo, do item anterior, em Verilog. Para isso, com o auxílio da tabela verdade do circuito, descrevemos todas as combinações das entradas A, B e Cin para as quais as saídas Cout e S são iguais a 1, ou seja, seus mintermos.

```
1  module mux8nova(A,B,Cin,Cout,S);
2  input A, B, Cin;
3  output Cout, S;
4
5  assign Cout = A & B & Cin |~A & B & Cin | A & ~B & Cin | A & B & ~Cin;
6  assign S = A & B & Cin | ~A & ~B & Cin |  A & ~B & ~Cin | ~A & B & ~Cin;
7  endmodule
```

Figura 7. Descrição comportamental do somador completo em Verilog

Em seguida, criamos um bloco utilizando a descrição comportamental realizada e desenhamos o diagrama esquemático do circuito.

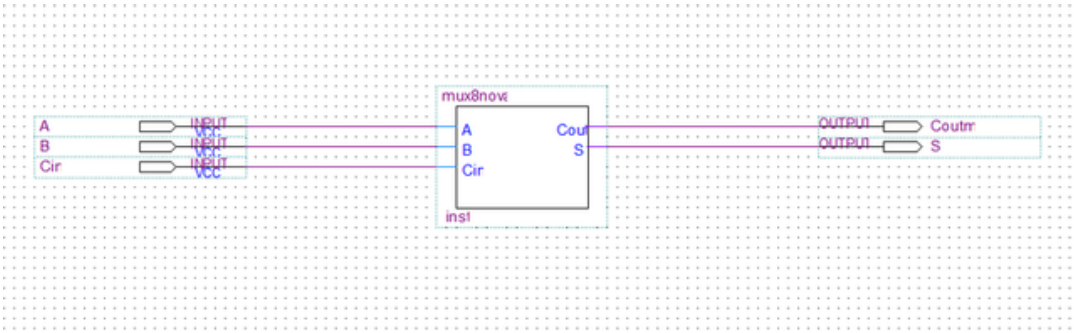

Figura 8. Diagrama esquemático do somador completo utilizando sua descrição comportamental em Verilog

Realizamos a compilação do circuito e obtivemos resultados positivos. Após a compilação bem sucedida, realizamos a simulação funcional em forma de onda para verificar o funcionamento da lógica do circuito.

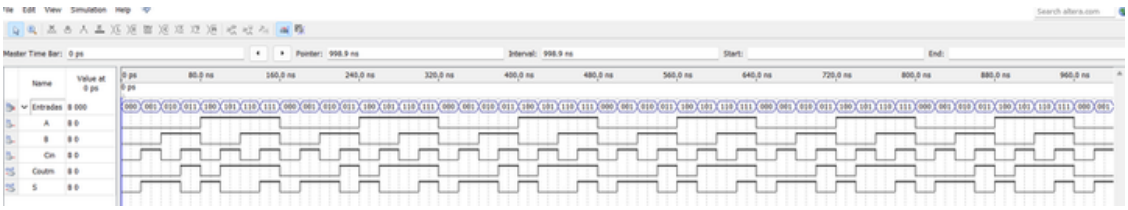

Figura 9. Simulação funcional em forma de onda do somador completo utilizando sua descrição comportamental em Verilog

Observando a forma de onda da simulação funcional, é possível concluir que a lógica do circuito está correta, visto que os resultados estão de acordo com a tabela verdade do somador.

Sabendo que a lógica do circuito está correta, procedemos com a simulação temporal em forma de onda.

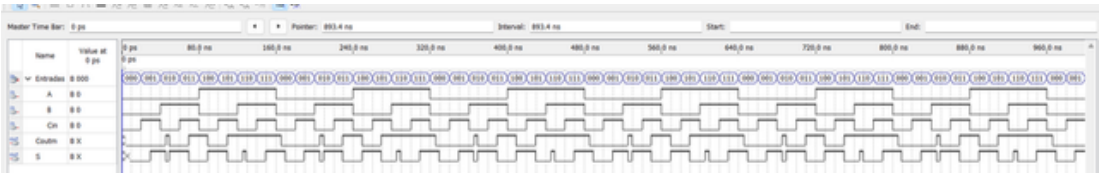

Figura 10. Simulação temporal em forma de onda do somador completo utilizando sua descrição comportamental em Verilog

Por fim, realizamos a pinagem do circuito e transferimos o mesmo para a placa FPGA DE2, como é possível verificar no vídeo. Obtivemos sucesso ao passar o nosso circuito para a placa; contudo, ela apresentou um erro inesperado. Apesar de todas as nossas simulações e códigos estarem corretos, ao acionarmos a entrada SW[2] (A) e deixarmos SW[1] (B) e SW[0] (Cin) em zero, a saída de Cout (LED R[1]) deveria ser zero, mas ela apresenta 1. Além disso, quando a combinação 101 é feita (SW2 em 1, SW1 em 0 e SW0 em 1), a saída de Cout deveria ser 1, mas apresenta zero. Não foi possível descobrir a origem desse erro, sendo considerado uma anomalia no sistema do Quartus II.

CONCLUSÃO

Neste experimento, detalhamos o processo de projeto, simulação e implementação de um somador completo utilizando multiplexadores. Através das etapas descritas, conseguimos criar, testar e configurar um MUX 8:1 para que ele refletisse os mintermos correspondentes às saídas do circuito. As simulações funcionais e temporais realizadas confirmaram que o somador estava funcionando corretamente, de acordo com sua tabela verdade. No entanto, ao transferir o circuito para a placa FPGA DE2 encontramos um erro nas saídas de Cout. Esta discrepância foi identificada, mas não sabemos sua origem, sendo considera uma anomalia do sistema. A implementação prática na FPGA destacou a importância de testar circuitos em hardware real para identificar possíveis falha s que não são aparentes nas simulações. Este experimento reforçou a necessidade de uma análise detalhada e a verificação contínua em todas as fases de desenvolvimento de circuitos digitais.

AUTO AVALIAÇÃO

5. Teste de Autoavaliação

Responda V ou F:

1. (V) Os multiplexadores e demultiplexadores são circuitos combinacionais.
2. (F) A tabela da verdade completa de um multiplexador com 2 terminais de seleção de dados tem 64 linhas.
3. (V) A tabela da verdade completa de um multiplexador com 8 terminais de entrada de dados tem 2^8 linhas.
4. (F) A tabela da verdade completa de um demultiplexador de 3 terminais de endereçamento tem 16 linhas e 8 colunas de saída.
5. (V) A tabela da verdade completa de um multiplexador tem exatamente o mesmo número de 0's e 1's.
6. (V) O demultiplexador é um decodificador com um terminal de ativação. Assim sendo, os terminais de endereçamento do demultiplexador correspondem aos terminais de entrada do decodificador.
7. (V) Continuando a questão anterior, o terminal de ativação do decodificador corresponde ao terminal de dados do demultiplexador.
8. (F) O multiplexador também é equivalente a um codificador.
9. (F) Um multiplexador com 1 terminal de seleção pode implementar qualquer uma das 16 funções de duas variáveis com apenas uma porta NOT adicional.
10. (V) Um multiplexador com 4 terminais de dados pode implementar qualquer função de 4 variáveis com apenas uma porta NOT adicional.
11. (F) Um multiplexador de 16 terminais de dados pode implementar qualquer função de 4 variáveis sem nenhuma porta adicional.